دليل شرح مشروع

PaySim Fraud Intelligence Platform

شرح عربي تفصيلي للعرض التقديمي والـDashboard

هذا الدليل مكتوب لتقدري تشرحي المشروع بنفسك: ما المشكلة؟ لماذا اخترنا هذا الموديل؟ كيف قرأنا النتائج؟ كيف تتحرك البيانات داخل الـDashboard؟ وما الذي لم ننفذه بعد؟

قبل العرض: احفظي القصة الأساسية للمشروع

المشروع ليس مجرد موديل يتوقع Fraud. المشروع عبارة عن Pipeline كاملة: نقرأ بيانات PaySim، نفهم سلوك العمليات، نجهز Features، ندرب مصنفًا، نختار Thresholds من Validation، نحوّل الدرجة إلى Risk Level وRecommended Action، نخزن النتيجة في SQL Server، ثم نعرضها في Dash حتى يستطيع المستخدم أن يبحث ويفلتر ويفحص العملية والتنبيه.

الجملة الذهبية نحن نبني نموذجًا تحليليًا لمراقبة الاحتيال بعد تنفيذ العملية على بيانات مالية اصطناعية، وليس نظامًا متصلًا ببنك حقيقي أو نظامًا يوقف الأموال فعليًا.

مفاهيم لازم تكون واضحة قبل الشرح

- Fraud Detection: محاولة التمييز بين عملية احتيالية وعملية طبيعية، والهدف `isFraud` قيمته 1 للاحتيال و0 للطبيعي.
- Class Imbalance: عدد عمليات Fraud قليل جدًا مقارنة بالطبيعي. لذلك Accuracy وحدها قد تكون مضللة.
- Feature: معلومة يدخلها الموديل، مثل amount أو log_amount أو balance behavior.
- Target: الإجابة التي نريد للموديل تعلمها، وهي isFraud. لا يجوز إدخالها ضمن X.
- Train: الجزء الذي يتعلم منه الموديل. Validation: نختار ونضبط عليه. Test: امتحان نهائي لا نستخدمه في اختيار Threshold.
- Raw Decision Score: رقم خام يخرج من decision_function. ليس نسبة احتمالية ولا Probability.
- Threshold: الحد الذي نقارن به الدرجة لاتخاذ قرار تصنيفي أو تشغيلي.
- Risk Policy: قواعد تحول الدرجة إلى LOW أو MEDIUM أو HIGH وإجراء مناسب.
- Dashboard: واجهة تفاعلية تقرأ SQL Server وتعرض البيانات من خلال Callbacks.

الشريحة 1 - عنوان المشروع

ابدئي بقول: مشروعنا اسمه PaySim Fraud Intelligence Platform. الفكرة أننا أخذنا معاملات مالية اصطناعية من PaySim، وبنينا عليها نظام تحليل ومراقبة للاحتيال. كلمة Intelligence تعني أننا لا نعرض رقمًا فقط؛ نحن نحول البيانات إلى Model Score ثم Risk Level ثم Alert، وبعد ذلك نخزن ونُظهر القرار في Dashboard. وضحي أن المشروع Prototype تعليمي على بيانات اصطناعية، وليس اتصالًا مباشرًا ببنك.

الشريحة 2 - ماذا يفعل النظام؟

اشرحي المراحل الثلاث: Detect أي اكتشاف العمليات التي تشبه Fraud، Explain أي إظهار الدرجة الخام ومستوى الخطر والإجراء المقترح، Monitor أي تخزين النتيجة في SQL Server وعرضها في Dash. أكدي أن Simulated Hold مجرد قيمة مسجلة للتوضيح، لا يوجد إيقاف دفع حقيقي ولا عكس أموال.

الشريحة 3 - المشكلة

قولي إن الاحتيال نادر، ولذلك الموديل يمكن أن يقول كل العمليات Normal ويحصل على Accuracy مرتفعة ظاهريًا، لكنه يفشل في اكتشاف Fraud. كمان False Positive يستهلك وقت فريق المراجعة، وFalse Negative يعني عملية احتيالية لم تُكتشف. لذلك نحتاج Precision وRecall وF1 وAverage Precision وConfusion Matrix، مش Accuracy فقط.

الشريحة 4 - التخطيط وأصحاب المصلحة

اشرحي أننا قسمنا العمل إلى مراحل واضحة: Data & EDA، ثم Feature Engineering، ثم Model، ثم Risk Engine، ثم Database، ثم Dashboard. المستخدم الأساسي المتوقع هو Fraud Analyst. الـEvaluator يحتاج دليلًا على المنهجية والاختبارات، والـDBA يحتاج مفاتيح وعلاقات وسلامة بيانات، والـML Engineer يهتم بمنع Leakage.

الشريحة 5 - البيانات

الملف الأصلي يحتوي 6,362,620 صفًا و11 عمودًا. كل صف يمثل معاملة. الأعمدة تشمل step، type، amount، identifiers وأرصدة قبل وبعد المعاملة، والهدف isFraud. عدد Fraud هو 8,213 فقط، أي نحو 0.1291%. يعني تقريبًا أقل من عمليتين احتيال لكل ألف معاملة. هذا يشرح صعوبة المسألة.

الشريحة 6 - اكتشافات EDA

اشرحي أن حالات Fraud في هذا الملف ظهرت فقط في TRANSFER وCASH_OUT. عندنا 4,097 تحويل و4,116 سحب نقدي. متوسط المبلغ في الحالتين أعلى في Fraud، لكن لا نقول إن كل مبلغ كبير احتيال. نحن نستخدم EDA لتوليد فرضيات ونختبرها، لا لإصدار حكم قطعي.

الشريحة 7 - الفرضيات الأربع

H1 المبلغ: متوسط Fraud أعلى. H2 الحساب الأصلي أُفرغ: النسبة مرتفعة جدًا في الاحتيال داخل النوعين. H3 Origin mismatch: النتيجة عكس توقع بسيط؛ mismatch أعلى بكثير في الطبيعي، لذلك لا نعتبره قاعدة احتيال. H4 Destination mismatch: قوي جدًا في TRANSFER لكنه ليس كذلك في CASH_OUT. الخلاصة أن تفسير Feature يعتمد على Transaction Type.

الشريحة 8 - التقسيم الزمني

قلّي لماذا لا نستخدم Random Split؟ لأننا نريد أن نتعلم من الماضي ونمتحن على المستقبل. التدريب من Step 1 إلى 520، Validation من 521 إلى 631، Test من 632 إلى 743. لاحظي أن Fraud Rate ارتفع من 0.0951% إلى 0.6173% ثم 1.3994%. هذا اسمه Distribution Shift، وهو تحذير أن الأداء يتغير مع الزمن.

الشريحة 9 - Data Leakage وتوقيت الخصائص

دي من أهم شرائح المشروع. عندنا newbalanceOrig وnewbalanceDest وهي أرصدة بعد المعاملة. المعلومة دي لا تكون متاحة قبل تنفيذ العملية، وبالتالي لو استخدمناها فنحن نحلل بعد التنفيذ. لهذا نسمي المنتج Post-transaction Monitoring. لو أردنا قرارًا قبل السماح بالعملية، سنعيد تصميم Feature Contract باستخدام المعلومات المتاحة قبل التنفيذ فقط.

الشريحة 10 - Feature Engineering

اشرحي أن Raw amount قد يكون منحرفًا جدًا، لذلك أنشأنا log_amount. وأنشأنا flags مثل orig_balance_emptied، وأخطاء ومطابقات الأرصدة. استخدمنا type كخاصية فئوية بعد encoding. احتفظنا بـnameOrig وnameDest للتتبع والبحث، لكن لم ندخلهما كميزات للموديل. isFraud هو Target وليس Feature، وisFlaggedFraud استُبعد من X.

الشريحة 11 - لماذا SGDClassifier؟

قولي إن الموديل هو SGDClassifier باستخدام "loss="log_loss. هو Linear Classifier يتعلم أوزان Features بواسطة Stochastic Gradient Descent. اخترناه لأنه مناسب لحجم بيانات كبير ويسمح بـpartial_fit على Batches. هو يشبه Logistic Regression من ناحية Logistic Loss لكنه ليس نفس كلاس LogisticRegression في Scikit-learn، وليس Random Forest أو Neural Network.

الشريحة 12 - الرياضيات

ابدئي من z = w transpose x + b. هذه Raw Margin. بعدها Sigmoid تحوّل z إلى p بين صفر وواحد. Logistic Loss تقيس خطأ الاحتمال بالنسبة للهدف y. SGD يحدث الأوزان كل Batch باتجاه يقلل Loss. الـLearning Rate يحدد حجم خطوة التحديث، والـRegularization يقلل تضخم الأوزان. StandardScaler يوحد مقاييس الخصائص حتى لا يهيمن Feature كبير الرقم مثل amount على التحسين.

الشريحة 13 - Class Weights

اشرحـي أن الطبيعي أكثر من 99.8% تقريبًا، ولو تركنا كل صف له نفس الأهمية قد يهيمن الطبيعي على Loss. Class Weight يرفع تأثير الفئة النادرة Fraud في التدريب. المعادلة الشائعة balanced weight هي N مقسومًا على K مضروبًا في عدد الفئة. لكن لا تذكري أرقام أوزان محددة قبل مراجعة قيمة class_weight داخل Training Script. الأهم أن الوزن يُحسب من Training فقط، ولا نستخدم Validation/Test لتحديده.

الشريحة 14 - Probability Saturation

دي أهم قصة فنية في المشروع. الـSigmoid لما يدخلها z كبير جدًا موجب تعطي قيمة قريبة جدًا من 1. لو قيم كثيرة مختلفة اتحولت في الكمبيوتر إلى 1.0 بالضبط، نفقد الترتيب بينها، وتظهر Ties. Average Precision يعتمد على ترتيب النتائج، فظهر AP من probabilities بقيمة 0.2046. لما استخدمنا decision_function حصلنا على Raw Margins حافظت على الترتيب وAP validation أصبح 0.995086. لذلك اعتمدنا Raw Score، لكن لا نسميه Probability.

الشريحة 15 - Thresholds والـRisk Policy

عندنا حد Medium يساوي 54.304175... وهو P99 لدرجات العمليات Normal في Validation. هذا اختيار تجريبي من الذيل الأعلى للـNormal، وليس احتمال Fraud 99% ولا سياسة بنك معتمدة. الحد High يساوي 213.808666... وتم اختياره على Validation لتعظيم F1. أقل Medium = LOW وMonitor. بينهما MEDIUM = Manual Review. أعلى High = HIGH وتنبيه ومحاكاة Hold.

الشريحة 16 - Validation

على Validation وباستخدام High Threshold: Precision 1.0، Recall 0.994068، F1 0.997025، و AP raw 0.995086. حصلنا على TP=1173 وFN=7 وFP=0. قولي إننا اخترنا Threshold على Validation فقط. هذه ليست النتيجة النهائية المستقلة؛ إنها مرحلة الاختيار والضبط.

الشريحة 17 - Final Test

بعد تثبيت Threshold، طبقناه مرة واحدة على Test: 89,466 عملية، منها 1,252 Fraud. TP=1236 وFN=16 وFP=0 عالية TN=88,214. Precision=100%، Recall=98.722%، F1=99.3569%، AP=98.9589%. Accuracy جدًا لكنها ليست المقياس الوحيد. صفر False Positives في هذه العينة لا يضمن صفرًا في الإنتاج.

الشريحة 18 - SQL Server

البيانات المحملة للـDashboard ليست كامل 6.36 مليون؛ هي Validation وTest فقط، 280,613 صفًا. تحفظ Transactions تحفظ تفاصيل العملية، FraudPredictions تحفظ Score وRisk وFraudAlerts وAction. عندنا 280,613 Transaction و280,613 Prediction و5,255 Alerts، ولا توجد duplicate في التنبيهات. SourceRowNumber ولا Orphan records. هذا يثبت سلامة العلاقات في الاختبار.

الشريحة 19 - Architecture

اشرحي التدفق من الشمال إلى اليمين: CSV ثم EDA وFeatures ثم Model ثم Risk Engine ثم SQL Server ثم Dashboard. كل طبقة لها مسؤولية محددة. الموديل لا يرسل تنبيهًا بنفسه؛ يعطي Score. Risk Engine يطبق Thresholds. Alert System يسجل التنبيه. SQL يحفظ، وDash يعرض.

الشريحة 20 - كيف يعمل Dash داخليًا؟

Dash عبارة عن Python Components. كل Component له id وproperties مثل value أو data أو figure. المستخدم يغير Dropdown، فيتغير value، فيطلق Input داخل Callback. الـCallback يستدعي دوال data_access. الدوال تفتح pyodbc connection وتنفذ SQL parameterized وتعيد Pandas DataFrame. Python يحوّل البيانات إلى Figure أو records، ثم Output يحدّث العنصر في المتصفح. هذا هو Reactive UI.

الشريحة 21 - الفلاتر

Dataset Split يختار ALL أو VALIDATION أو TEST. Type Filter يختار في Transaction Type. Risk Filter النسخة الحالية يدخل في Type Analysis وTransaction Explorer؛ لا يدخل في KPI Query لأن fetch_kpis لا تستقبله. Refresh Button تحديث يدوي، وdcc.Interval كل 30 ثانية. البحث يستهدف رقم الصف أو اسم المرسل أو المستلم. لازم تكوني دقيقة في توضيح نطاق كل فلتر.

الشريحة 22 - الرسوم

الرسم الأول يقارن Actual Fraud وPredicted Fraud بالأعمدة، ويعرض Transaction Volume بخط على محور ثانوي لأن المقياس مختلف. الدونات تعرض عدد العمليات داخل Risk Tiers، وليست Probability. الرسم الزمني يعرض حجم المعاملات بالأعمدة، Actual وPredicted Fraud بخطوط، ومتوسطًا متحركًا 7 Steps لتخفيف التذبذب.

الشريحة 23 - Transaction Explorer

المستخدم يبحث عن SourceRowNumber أو Sender ID أو Receiver ID. الاستعلام يعيد 100 صف كحد افتراضي. نعرض 14 عمودًا. عند اختيار صف، Callback التفاصيل يستخدم State من بيانات الجدول ويرسم Panel فيه النوع والمبلغ والمرسل والمستلم وRaw Decision Score وRisk Level وRecommended Action وAlert. لاحظي أن Amount كانت Decimal وتحولت إلى float قبل إرسالها للواجهة.

الشريحة 24 - Alert Center

مركز التنبيهات يقرأ FraudAlerts من SQL. الفلاتر: Alert Status، Transaction Type، Dataset Split. النتيجة 15 عمودًا، والحد 100. تاريخ الإنشاء يتم تحويله إلى ISO string، وDecimal إلى float، وBool إلى YES/NO. يوجد تحديث يدوي وكل 30 ثانية. هذا جدول تنبيهات محفوظ، وليس Notification حقيقي للمستخدم.

الشريحة 25 - أدلة التنفيذ

اذكري الاختبارات التي نفذت: فحص Parquet، Risk Engine Smoke Test، Alert Routing، SQL Integrity، Full Integration، Alert Center، Transaction Explorer، Analytics Callback، Charts. التكامل النهائي أعطى 4 Callbacks و23 IDs ولا يوجد Missing IDs أو Duplicate Outputs. التطبيق فتح محليًا على 127.0.0.1:8050 وظهرت HTTP 200 responses.

الشريحة 26 - الحدود والمسؤولية

PaySim بيانات اصطناعية، والنتائج لا تعمم تلقائيًا على بنك حقيقي. هناك Distribution Shift. Raw Margin ليست Probability. Medium Threshold تجريبية. الخصائص اللاحقة تجعل النظام Post-transaction. لا يوجد Real

Drift monitoringو Human reviewو Cost-based thresholdsو Calibrationو Real-world validation نحتاج Hold. قبل الإنتاج.

الشريحة 27 - SQL Injection وMITM المستقبلية

قولي إن هذه ليست جزءًا من موديل Fraud الحالي. مرحلة لاحقة منفصلة. SQL Injection Lab ستعرض Query غير آمنة في بيئة محلية آمنة، ثم Parameterized Query والحماية والاختبار. MITM Lab ستكون Simulation محلية لاعتراض أو تعديل رسالة، ثم التحقق من HMAC أو Signature واكتشاف التلاعب. لا يوجد اختراق شبكة حقيقية.

الشريحة 28 - GitHub والـCloud

GitHub يخزن Source Code وSQL Scripts وDocs. Cloud Host يشغل تطبيق Dash على الإنترنت. لكن قاعدة SQL Server الموجودة على جهازك لا تصبح متاحة تلقائيًا للسيرفر السحابي. نحتاج Remote SQL Endpoint أو نقل البيانات إلى قاعدة سحابية، وتغيير Windows Authentication إلى Environment Variables / Secret-based authentication، وتشغيل Gunicorn أو Docker. لا نضع Password داخل GitHub.

الشريحة 29 - الخاتمة

اختصري القصة: حولنا بيانات اصطناعية كبيرة وغير متوازنة إلى Model قابل للتقييم الزمني، واستخدمنا Raw Margin لمعالجة مشكلة saturation، وحولنا Score إلى Risk Policy، وخزّنا النتيجة في SQL Server، وبنينا Dashboard للمتابعة والفحص. المرحلة التالية تحسين السياسة والاختبار على بيانات واقعية المسموح بها ثم الأمن السيبراني والنشر السحابي.

إجابات أسئلة متوقعة من لجنة التقييم

لماذا لم تستخدمي Accuracy فقط؟

لأن Fraud نادر جدًا. توقع كل الصفوف Normal يمكن أن يعطي Accuracy تقارب 99.87% دون اكتشاف حالة Fraud واحدة. لذلك استخدمنا Precision وRecall وF1 وAP وConfusion Matrix.

لماذا SGDClassifier وليس LogisticRegression؟

لأنه يدعم التدريب التدريجي partial_fit على Batches ومناسب لبيانات كبيرة. loss=log_loss يجعله مصنفًا خطيًا بخسارة لوجستية.

ما معنى weight؟

وزن الفئة يضرب مساهمة العينات في Loss. رفع وزن Fraud يجعل الخطأ على الفئة النادرة أكثر تأثيرًا خلال التدريب. لا أذكر القيمة الرقمية إلا بعد مراجعة إعداد التدريب.

ما معنى saturation؟

Sigmoid تقترب من 1 مع z موجب كبير. إذا انهارت درجات كثيرة إلى 1.0، تصبح متساوية رقميًا، ونفقد ترتيبها. decision_function يحتفظ بالـraw margin.

هل Raw Score يعني 95% احتمال؟

لا. هو margin غير معاير. يجب عدم قراءته كنسبة احتمالية.

لماذا Medium Threshold عند P99؟

لإنشاء ذيل أعلى تجريبي من درجات Normal على Validation لإحالة عدد محدود نسبيًا للمراجعة، لكنه ليس قرار Business معتمدًا ويحتاج ضبطًا بحسب تكلفة المراجعة.

لماذا Post-transaction؟

لأن بعض الخصائص تعتمد على newbalance بعد العملية. قبل التنفيذ هذه المعلومات غير متاحة.

هل النموذج أوقف تحويلات؟

لا. النظام يسجل Recommended Action وSimulatedHold فقط في بيئة PaySim تجريبية.

لماذا لا يعمل Cloud مباشرة؟

لأن التطبيق المحلي يستخدم Windows Authentication وقاعدة SQL Server على جهاز المطور. Cloud Linux process لا يستطيع رؤية D: أو استخدام Windows Login المحلي. نحتاج قاعدة بعيدة واتصالًا مؤمنًا.

ماذا ستفعلين لاحقًا؟

تحسين Thresholds وCalibration وDrift Monitoring وHuman Review، ثم SQL Injection Defensive Lab وMITM Integrity Simulation، ثم Cloud Deployment بعد تجهيز قاعدة بعيدة.

طريقة شرح الـDashboard أمام الشاشة - خطوات عملية

1. افتحي الصفحة الرئيسية

قولي: هذه واجهة تنفيذية. أول جزء يوضح اسم المنصة وحالة الاتصال بقاعدة PaySimFraudDB.

2. اشرحي الفلاتر

اختاري TEST. لاحظي أن KPIs تتغير إلى إحصاءات Test. بعد ذلك اختاري CASH_OUT. ثم اختاري HIGH. اشرحي أن كل فلتر يصل إلى الاستعلامات التي تدعمه، وليس بالضرورة كل KPI.

3. اشرحي KPIs

Total Transactions حجم العينة المختارة. Actual Fraud هو عدد الملصقات التاريخية الحقيقية المخزنة لأغراض التقييم. High Risk هو HIGH. Manual Review هو عدد صفوف Fraud Prevalence = Actual Fraud / Total. F1 في الشاشة الحالية محسوب من نطاق استعلام KPI الحالي، لا تخلطيه مع Final Test F1 إلا بعد اختيار TEST. MEDIUM.

4. اشرحي Fraud by Type

قارني العمود الفعلي بالحالات التي توقعها النموذج. الخط هو Total Volume على محور ثانوي.

5. اشرحي Risk Donut

هذا عدد العمليات في كل Tier، وليس نسبة احتمال Fraud.

6. اشرحي Temporal Chart

العمود يمثل حجم المعاملات، والخطوط تمثل Fraud حقيقي ومكتشف، والخط الثالث متوسطًا متحركًا.

7. Transaction Explorer

ابحثي برقم Source Row، أو اختاري عملية. بعد اختيار صف تفتح بطاقة التفاصيل. وضحي Raw Score و Risk Level وAction.

8. Alert Center

اختاري High Risk أو Open ووضحي التنبيه ووقت إنشاءه والعملية المرتبطة به.

9. Refresh

اضغطي Refresh Data. وضحي أن هناك أيضًا تحديث تلقائي كل 30 ثانية. لا يلزم أن تتغير القيم إذا قاعدة البيانات ثابتة؛ التحديث يعيد القراءة فقط.

تسلسل عرض حي مدته 3 إلى 4 دقائق

- 30 ثانية: المشكلة ونسبة Fraud المنخفضة جدًا.
- 40 ثانية: EDA والفرضيات ثم التقسيم الزمني.
- 50 ثانية: SGDClassifier وClass Weights وSaturation وRaw Score.
- 35 ثانية: Validation Thresholds ونتيجة Final Test.
- 60 ثانية: Dashboard - الفلاتر والمؤشرات والرسوم والبحث والتنبيهات.
- 20 ثانية: حدود النظام والمرحلة التالية SQLi وMITM وCloud.

تنبيه مهم جدًا عن أعمدة الأرصدة في PaySim

في بيانات Kaggle الخاصة بـPaySim توجد ملاحظة صريحة: المعاملات التي يتم اكتشافها كاحتيال تُلغى، ولذلك ينصح مصدر البيانات بعدم استخدام oldbalanceOrg وnewbalanceOrig وoldbalanceDest وnewbalanceDest في اكتشاف الاحتيال. نحن استخدمنا في التجربة الحالية خصائص مشتقة من الأرصدة، ولذلك يجب أن أقول للجنة بوضوح إن النتائج الحالية تحتاج اختبارًا إضافيًا بدون أي Balance Features. هذا الاختبار يسمى Ablation Study: نعيد التدريب بنفس التقسيم الزمني ونفس قواعد اختيار Threshold، لكن نحذف أعمدة الأرصدة الخام وكل الخصائص المشتقة منها، ثم نقارن النتائج. لا أخفي هذه الملاحظة ولا أقدم نتيجة الموديل الحالي كأنها نهائية للإنتاج.

سؤال متوقع: لماذا استخدمنا الأرصدة رغم تحذير المصدر؟

اختبرنا سلوك الأرصدة كما يظهر في البيانات، وبنينا Feature Engineering وEDA الإجابة الدقيقة: في مرحلة تجعل هذه الخصائص موضع شك في الصلاحية. لذلك لا أبرر Kaggle نموذجًا تجريبيًا بعد المعاملة. لكن ملاحظة استخدامها للإنتاج؛ أعتبرها تجربة أولى تحتاج إعادة تدريب ومقارنة مستقلة بدون الأرصدة قبل الحكم على قوة النموذج.